TP N°1 : Algorithme d'Euclide Étendu et Chiffrement Affine

Master 1 - Informatique

Février 2026

Réalisé par :
Bentrat Zineb

Groupe : ISI G1

Année universitaire : 2025/2026

1 Introduction

Ce TP porte sur l'implémentation de l'algorithme d'Euclide étendu et son application directe dans la cryptographie classique via le chiffrement affine. L'objectif est de maîtriser l'inversion modulaire pour garantir la réversibilité des algorithmes de chiffrement.

2 Algorithme d'Euclide Étendu

L'algorithme d'Euclide étendu permet de calculer le plus grand commun diviseur (PGCD) de deux entiers a et b, ainsi que les coefficients de Bézout u et v tels que $au + bv = gcd(a, b)$.

2.1 Implémentation en Python

```python
def extended_gcd(a, b):
    r, r_prime = a, b
    u, u_prime = 1, 0
    v, v_prime = 0, 1

    while r_prime != 0:
        q = r // r_prime
        r, r_prime = r_prime, r - q * r_prime
        u, u_prime = u_prime, u - q * u_prime
        v, v_prime = v_prime, v - q * v_prime
    return r, u, v
```

3 Inversion Modulaire

Un entier a possède un inverse modulo n s'il existe un entier x tel que $ax \equiv 1 \pmod{n}$. Selon l'identité de Bézout, si $gcd(a, n) = 1$, il existe u et v tels que $au + nv = 1$. En appliquant le modulo n, nous obtenons l'égalité suivante :

$$au \equiv 1 \pmod{n} \tag{1}$$

Ainsi, la valeur de $u \pmod{n}$ constitue l'inverse modulaire recherché.

```python
def mod_inverse(a, n):
    r, u, v = extended_gcd(a, n)
    if r != 1:
        # Signalement de l'absence d'inverse
        return None
    return u % n
```

4 Chiffrement Affine

Le chiffrement affine est une méthode de substitution mono-alphabétique définie par la fonction $E_{a,b}(m) = (am + b) \pmod{n}$. Pour que ce chiffrement soit réversible (déchiffrable), la clé a doit impérativement être première avec la taille de l'alphabet n.

4.1 Fonctions de Chiffrement et Déchiffrement

Le déchiffrement utilise l'inverse modulaire de a pour retrouver le message original : $D_{a,b}(c) = a^{-1}(c - b) \pmod{n}$.

```python
def affine_encrypt(m, a, b, n=26):
    return (a * m + b) % n

def affine_decrypt(c, a, b, n=26):
    a_inv = mod_inverse(a, n)
    if a_inv is not None:
        return (a_inv * (c - b)) % n
```

5 Résultats et Conclusion

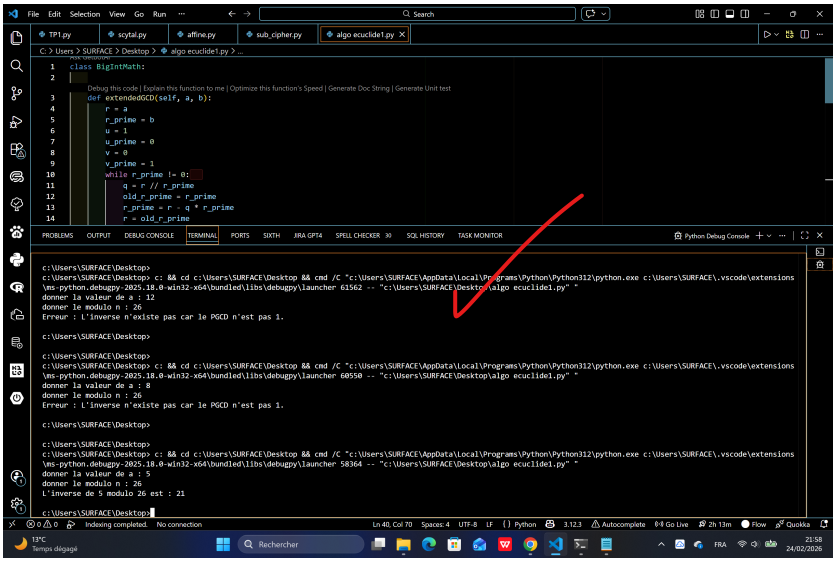

Figure 1: Capture d'écran de l'exécution du programme dans le terminal

Nous avons vérifié expérimentalement que l'application successive du chiffrement et du déchiffrement redonne le message initial. Ce TP démontre que l'inversion modulaire via l'algorithme d'Euclide étendu est la pièce maîtresse permettant la réversibilité du chiffrement affine.

1/2